

Part 1: Variables (Questions 1 - 10)

Question 1: What is a variable in C/C++?

A variable is a named storage location in memory used to hold data that can be modified during program execution. It acts as a container for data values, mapping a human-readable identifier to an actual physical memory address.

Question 2: Why do we use variables in programs?

We use variables to store, retrieve, and manipulate data dynamically. Without variables, runtime data like user input, calculation steps, or application state couldn't be saved, making dynamic software processing impossible.

Question 3: What is the difference between a variable name and its value?

- **Variable Name (Identifier):** The unique text label used in source code to reference the allocated memory block (e.g., *age*).
- **Variable Value:** The actual binary data content stored inside that specific memory address location (e.g., *18*).

Question 4: Which of the following is a valid variable name?

Options: a) 2num b) total_marks c) for d) my marks

Correct Answer: *b) total_marks*

Reasoning: Identifiers cannot start with digits (*2num*), cannot be keywords (*for*), and cannot contain whitespaces (*my marks*). Underscores are fully valid characters.

Question 5: Why is Age different from age?

C and C++ are strictly **case-sensitive** programming languages. The compiler reads uppercase and lowercase characters as separate ASCII/Unicode tokens, creating completely individual allocations for *Age* and *age*.

Question 6: Can a variable name contain spaces? Why?

No, a variable name cannot contain spaces. Whitespace characters act as token delimiters in the language syntax rules. Including a space causes the compiler to interpret the input as two disconnected expressions, triggering a syntax error.

Question 7: Can a variable name start with an underscore (_) ?

Yes. A variable name can start with an underscore (e.g., *_total*). It follows identical rules to letters. However, starting with underscores is typically discouraged unless developing system code, as internal compiler and standard library variables often utilize them.

Question 8: What happens if two variables have the same name in the same scope?

The compiler will issue a "**redefinition / redeclaration error**". Because identical identifiers in the same scope point to the exact same lookup namespace, the compiler cannot uniquely resolve which memory track or type declaration to manipulate.

Question 9: Write three meaningful variable names for storing student information.

- *studentRollNumber*
- *studentGpa*
- *studentEmailAddress*

Question 10: Which naming convention is better and why: studentMarks or sm?

Answer: *studentMarks* is significantly better.

Why: Self-documenting descriptive names convey clear meaning immediately to any reader or developer, decreasing maintenance costs and context errors. Cryptic shorthand like *sm* forces readers to hunt for definitions.

Part 2: Data Types (Questions 11 - 20)

Question 11: What is a data type?

A data type is an architectural property that tells the compiler what category of information a variable holds, how many bytes of memory to allocate, and what operations are valid to perform on that sequence of bits.

Question 12: Which data type is used to store whole numbers?

The *int* (Integer) primitive data type is standard for representing non-fractional integers.

Question 13: Which data type is used to store decimal values?

The floating-point primitive data types, specifically *float* or *double*, are used to store numbers with fractions.

Question 14: What is the difference between float and double?

- **float:** Uses 4 bytes of memory, offering single-precision resolution (~7 decimal accuracy digits).
- **double:** Uses 8 bytes of memory, offering double-precision resolution (~15 decimal accuracy digits). It is the default for floating literals in C++.

Question 15: Which data type stores a single character?

The *char* primitive data type, which stores a single 1-byte character literal bounded by single quotes.

Question 16: Which data type stores True/False values?

The *bool* (Boolean) data type, resolving fundamentally to binary states true (1) or false (0).

Question 17: What data type is suitable for storing a person's name?

The *std::string* class from the standard library is best suited in C++. (In legacy C, an array of characters *char[]* is utilized).

Question 18: Predict the output: `int age=18; cout<<age;`

Output: 18

Question 19: Identify the data type: 25, 3.14, 'A', true.

- 25 → *int*
- 3.14 → *double*
- 'A' → *char*
- true → *bool*

Question 20: What happens if you store a decimal number in an int variable?

The runtime performs an **implicit truncation conversion**. The fractional decimal component is completely discarded (chopped off) instead of being rounded up or down. For instance, assigning 5.99 to an integer yields a value of exactly 5.

Part 3: Input & Output (Questions 21 - 30)

Question 21: What is the purpose of `cin`?

cin (standard character input object) is an instance connected to the keyboard or input subsystem, used to capture data written by users into targeted variables.

Question 22: What is the purpose of cout?

cout (standard character output object) is an instance linked directly to the primary console display screen, used to send stream items to viewable space.

Question 23: Which symbol is used with cin?

The stream extraction operator: >>

Question 24: Which symbol is used with cout?

The stream insertion operator: <<

Question 25: Explain: cin >> age;

The computer halts sequential line operations, listening for keystrokes on the standard terminal. When a sequence is typed followed by Enter, the processor reads that string segment, transforms it to match the integer format, and updates the value at the memory space reserved for variable *age*.

Question 26: Explain: cout << age;

The engine queries the memory lookup table for the identifier *age*, reads its stored value, pushes it through the console stream pipeline, and displays the literal content visually inside the interface console terminal.

Question 27: What is the purpose of endl?

endl is an explicit stream manipulator that outputs a newline control block () and instantly triggers an explicit execution flush on the output stream's system internal buffer to force layout delivery.

Question 28: Write a statement to input a student's marks.

```
cin >> studentMarks;
```

Question 29: Write a statement to display 'Welcome to C++'.

```
cout << "Welcome to C++";
```

Question 30: Predict the output: `cout << 'Programming';`

Result: Compilation Error!

Explanation: Text strings must use standard double-quotes ("..."). Single quotes ('...') define a single multi-character integer constant, causing a compilation overflow error or undefined token matching.

Part 4: Operators (Questions 31 - 40)

Question 31: What is an operator?

An operator is a unique symbolic sequence or token that directs the compiler engine to execute precise mathematical, relational logic, or system assignments on variables and value configurations (known as operands).

Question 32: Which operator is used for addition?

The arithmetic addition token: +

Question 33: What is the result of `10 % 3`?

Result: 1

Explanation: The modulo arithmetic operator evaluates the remainder of long division. 3 goes into 10 three full times, leaving a fractional residue of exactly 1.

Question 34: What is the result of $8 \% 2$?

Result: *0* (Because 8 handles integer split perfectly, leaving zero remainder items).

Question 35: Which operator checks equality?

The comparison / relational equality operator: `==`

Question 36: What is the difference between `=` and `==` ?

- `=` is the **assignment operator**. It copies the processed calculation value from the right hand side and binds it inside the variable block name listed on the left.
- `==` is the **relational operator**. It tests whether values on both sides match identically, resolving to a boolean result of true or false.

Question 37: What is the result of $5 > 3$?

Result: *true* (or evaluates explicitly to integer constant *1* inside computational logic processing checks).

Question 38: What does `&&` represent?

The short-circuiting **Logical AND** operator. It validates logical statements on both sides, returning true if and only if both individual expressions evaluate to true.

Question 39: What does `||` represent?

The short-circuiting **Logical OR** operator. It evaluates paired criteria and returns true if at least one of the conditions validates to a true outcome.

Question 40: What does ! represent?

The unary **Logical NOT** operator (Negation). It reverses the active logical state of a boolean operand, converting a true expression to false, and a false expression to true.

Part 5: Basic Programming Problems (Questions 41 - 50)

Question 41: Print Hello World

Component	Details
1. Problem Statement	Write a program to display the basic literal text string "Hello World" onto the standard computer monitor interface.
2. Algorithm	1. Start. 2. Route string text literal "Hello World" down the stream channel to <i>cout</i>. 3. Finish process execution routines.
3. Dry Run	Input: <i>None</i> Stream Evaluation: Pipeline receives "Hello World" → Output buffer loads text data → Screen updates.
4. Program Code	<pre>#include <iostream> using namespace std; int main() { cout << "Hello World" << endl; return 0; }</pre>
5. Expected Output	Hello World
6. Logic Explanation	The program includes the basic input-output header framework stream. Using the global context <code>std::cout</code> , the left-shift insertions link our standard string literal layout text data to screen endpoints cleanly.

Question 42: Sum of Two Numbers

Component	Details
1. Problem Statement	Write a program to input two integers from a user terminal sequence, evaluate their sum, and output that calculation total.
2. Algorithm	

	1. Start. 2. Instantiate storage space containers: <i>num1</i>, <i>num2</i>, and <i>sum</i>. 3. Read keyboard parameters directly into variables <i>num1</i> and <i>num2</i>. 4. Calculate sum arithmetic operation: <i>sum</i> = <i>num1</i> + <i>num2</i>. 5. Render the calculated <i>sum</i> variable back out via display layouts.
3. Dry Run	Inputs tracked: <i>num1</i> = 12, <i>num2</i> = 8 Computation: <i>sum</i> = 12 + 8 → Total variable evaluated holds value 20.
4. Program Code	<pre>#include <iostream> using namespace std; int main() { int num1, num2, sum; cout << "Enter two numbers: "; cin >> num1 >> num2; sum = num1 + num2; cout << "Sum: " << sum << endl; return 0; }</pre>
5. Expected Output	<pre>Enter two numbers: 12 8 Sum: 20</pre>
6. Logic Explanation	The extraction operator parses numerical entries safely from standard system input sequences. Arithmetic binary engine rules use code flags to build calculations inside memory registers.

Question 43: Area of a Rectangle

Component	Details
1. Problem Statement	Write a program to dynamically accept values for length and width from users, compute the comprehensive area coverage, and print out the metric result.
2. Algorithm	1. Start. 2. Setup variables with floating point properties: <i>length</i>, <i>width</i>, <i>area</i>. 3. Capture dimension arguments safely into <i>length</i> and <i>width</i> variables. 4. Run calculation structure formula: <i>area</i> = <i>length</i> × <i>width</i>. 5. Print output values for tracking metrics.
3. Dry Run	

	Input Parameters: <i>length</i> = 5.5, <i>width</i> = 4.0 Math processing sequence steps: <i>area</i> = 5.5 × 4.0 = 22.0.
4. Program Code	<pre> #include <iostream> using namespace std; int main() { double length, width, area; cout << "Enter length and width: "; cin >> length >> width; area = length * width; cout << "Area of rectangle: " << area << endl; return 0; } </pre>
5. Expected Output	<pre> Enter length and width: 5.5 4 Area of rectangle: 22 </pre>
6. Logic Explanation	Double precision components handle dimensions with arbitrary fractional details. Multiplication parameters use asterisks to trigger high-speed standard operations inside hardware float structures.

Question 44: Formula for Simple Interest

The operational equation formulation utilized globally is:

$$\text{Simple Interest} = (P \times R \times T) / 100$$

Where properties mean: *P* = Principal primary capital amount; *R* = Rate percent multiplier factor per year interval; *T* = Time tracking duration specified in uniform calendar years.

Question 45: Simple Interest Calculation Program

Component	Details
1. Problem Statement	Write a complete execution routine calculating dynamic financial simple interest tracking from parameters assigned at runtime.
2. Algorithm	1. Start. 2. Prepare tracking spaces for allocation: <i>principal</i> , <i>rate</i> , <i>time</i> , <i>si</i> .

	3. Extract transaction profiles into target input positions. 4. Evaluate logic parameters: $si = (principal \times rate \times time) / 100.0$. 5. Return calculation records back outward.
3. Dry Run	Inputs tracked: $principal = 1000.0$, $rate = 5.0$, $time = 2.0$ Steps resolved: $si = (1000 \times 5 \times 2) / 100.0 \rightarrow 10000.0 / 100.0 = 100.0$.
4. Program Code	<pre>#include <iostream> using namespace std; int main() { double principal, rate, time, si; cout << "Enter Principal, Rate, and Time: "; cin >> principal >> rate >> time; si = (principal * rate * time) / 100.0; cout << "Simple Interest calculated: " << si << endl; return 0; }</pre>
5. Expected Output	<pre>Enter Principal, Rate, and Time: 1000 5 2 Simple Interest calculated: 100</pre>
6. Logic Explanation	Dividing specifically by explicit floating decimal configuration constants stops compiler routines from applying incorrect truncation steps on interest returns.

Question 46: Why is a temporary variable used while swapping two numbers?

A temporary variable operates as an isolated intermediate data holding area. In a sequential processor line, executing direct tracking overwrite steps such as $a = b$ destroys original record details inside memory position a instantly. Using an intermediate placeholder acts as a backup buffer, allowing safe non-destructive shuffling of information.

Question 47: Swap Two Variables

Component	Details
1. Problem Statement	Construct a code routine exchanging data points bounded inside two unique isolated variable spaces safely without loss.
2. Algorithm	

	1. Start. 2. Allocate variables: $a = 10$, $b = 20$, plus empty proxy block <i>temp</i>. 3. Cache values safely away: $temp = a$. 4. Execute target shifts safely: $a = b$. 5. Transfer cached states back to place: $b = temp$. 6. Print outputs to verify states.
3. Dry Run	Initial: $a=10, b=20$ Step 1: $temp = 10$ Step 2: $a = 20$ Step 3: $b = 10 \rightarrow$ Swap operation verified complete.
4. Program Code	<pre>#include <iostream> using namespace std; int main() { int a = 10, b = 20; cout << "Pre-swap states: a = " << a << ", b = " << b << endl; int temp = a; a = b; b = temp; cout << "After swap: a = " << a << ", b = " << b << endl; return 0; }</pre>
5. Expected Output	Pre-swap states: a = 10, b = 20 After swap: a = 20, b = 10
6. Logic Explanation	The 3-step shifting algorithm handles complex assignments by using an auxiliary tracking pointer to swap values cleanly without mathematical overhead or data loss.

Question 48: Formula to Convert Celsius into Fahrenheit

The standard algebraic conversation rule identity formula mapping is:

$$F = (C \times 9/5) + 32$$

In computational code configurations, developers typically use the expression form: $F = (C \times 1.8) + 32$

Question 49: Fahrenheit value of 0°C

Result: 32°F

Step-by-step Evaluation Check:

$$F = (0 \times 1.8) + 32 \Rightarrow F = 0 + 32 \Rightarrow F = 32$$

Question 50: Integer Division Risks in Scaling Conversions

Component	Details
1. Problem Statement	Analyze and resolve systemic precision errors caused by utilizing standard integer division metrics within the calculation statement formulas for temperature parsing.
2. Explanation of Logic & Cause	In C/C++, executing mathematical operations between two strict integer elements forces the engine to resolve the final response as an integer tracking component. The fraction equation element $9/5$ mathematically equals 1.8. However, when processed under strict integer criteria rules, the compiler completely truncates and drops the fractional trailing items, calculating the sub-step as exactly 1. Multiplying Celsius inputs by 1 instead of 1.8 breaks calculation tracking completely. Writing literal elements as floating point definitions (e.g., $9.0/5.0$) forces the compiler to use float processing, keeping the calculation accurate.
3. Correct Code Fix	<pre>#include <iostream> using namespace std; int main() { double celsius, fahrenheit; cout << "Enter temperature in Celsius: "; cin >> celsius; fahrenheit = (celsius * 9.0 / 5.0) + 32.0; cout << "Fahrenheit equivalent: " << fahrenheit << endl; return 0; }</pre>